

SQL to PySpark Phase 2 – Revised Bridge Pack

Reading the CUSTOMERS AND SALES DATA

```
In [ ]: df=spark.read.format("csv").option("header",True).option("inferSchema",True).option("delimiter",",").load("/Volumes/workspace/default/customers")
df.display()
```

customer_id	first_name	last_name	email
1	John	Smith	john.smith@domain.com
2	Emma	Jones	emma.jones@webmail.com
3	Olivia	Brown	olivia.brown@outlook.com
4	Liam	Johnson	liam.johnson@gmail.com

```
In [ ]: df=spark.read.format("csv").option("header",True).option("inferSchema",True).option("delimiter",",").load("/Volumes/workspace/default/sales")
df.display()
```

Item_Identifier	Item_Weight	Item_Fat_Content	Item_Visibility	Item_Outlet_Identifier
FDA15	9.3	Low Fat	0.016047301	Dairymart_Pune
DRC01	5.92	Regular	0.019278216	Soft_Banarsidas
FDN15	17.5	Low Fat	0.016760075	Mart_Banarsidas
FDX07	19.2	Regular	0.0	Fruit_Veg_Banarsidas
NCD19	8.93	Low Fat	0.0	HCC_Banarsidas

```
In [ ]: orders = spark.read.format("csv").option("header",
True).option("inferSchema", True).option("delimiter",
",").load("/Volumes/workspace/default/customers/orders.csv")
display(orders)
orders.printSchema()
```

order_id	customer_id	order_date	status	total_amount
1	1	2024-01-15	Shipped	39.98
2	1	2024-01-16	Pending	29.99
3	2	2024-01-17	Delivered	25.0
4	2	2024-01-18	Shipped	89.97
5	3	2024-01-19	Pending	49.98
6	3	2024-01-20	Delivered	119.96
7	4	2024-01-21	Shipped	15.5
8	4	2024-01-22	Pending	66.75

```
root
|-- order_id: integer (nullable = true)
|-- customer_id: integer (nullable = true)
|-- order_date: date (nullable = true)
|-- status: string (nullable = true)
|-- total_amount: double (nullable = true)
```

```
In [ ]: customers = spark.read.option("header",
"true").csv("/Volumes/workspace/default/customers")
orders = spark.read.option("header",
"true").csv("/Volumes/workspace/default/customers/orders.csv")
display(customers)
display(orders)
customers.printSchema()
orders.printSchema()
```

customer_id	first_name	last_name	email
1	John	Smith	john.smith@domain.com
2	Emma	Jones	emma.jones@webmail.com
3	Olivia	Brown	olivia.brown@outlook.com
4	Liam	Johnson	liam.johnson@gmail.com

order_id	customer_id	order_date	status	total_amount
1	1	2024-01-15	Shipped	39.98
2	1	2024-01-16	Pending	29.99
3	2	2024-01-17	Delivered	25.0
4	2	2024-01-18	Shipped	89.97
5	3	2024-01-19	Pending	49.98
6	3	2024-01-20	Delivered	119.96
7	4	2024-01-21	Shipped	15.5
8	4	2024-01-22	Pending	66.75

```
root
|-- customer_id: string (nullable = true)
|-- first_name: string (nullable = true)
|-- last_name: string (nullable = true)
|-- email: string (nullable = true)
|-- phone_number: string (nullable = true)
|-- address: string (nullable = true)
|-- city: string (nullable = true)
|-- state: string (nullable = true)
|-- zip_code: string (nullable = true)
```

```
root
```

```
|-- order_id: string (nullable = true)
|-- customer_id: string (nullable = true)
|-- order_date: string (nullable = true)
|-- status: string (nullable = true)
|-- total_amount: string (nullable = true)
```

REMOVING NULL COLUMNS

```
In [ ]: # Before solving the exercises, remove rows with missing customer_id
        from both datasets.
        customers = customers.dropna(subset=["customer_id"])
        orders = orders.dropna(subset=["customer_id"])
```

1. Total order amount for each customer

```
In [ ]: from pyspark.sql.functions import sum

        total_order_amount =
        orders.groupBy("customer_id").agg(sum("total_amount").alias("total_order_
        _amount"))
        display(total_order_amount)
```

customer_id	total_order_amount
14	89.97
16	119.96
35	49.98
3	169.94
8	169.94
12	29.99
20	110.95
27	15.5

2. Top 3 customers by total spend

```
In [ ]: from pyspark.sql.functions import sum

top_3_customers =
orders.groupBy("customer_id").agg(sum("total_amount").alias("total_spend
")).orderBy("total_spend", ascending=False).limit(3)
display(top_3_customers)
```

customer_id	total_spend
3	169.94
8	169.94
10	150.95

3. Customers with no orders

```
In [ ]: customers_no_orders = customers.join(orders, on="customer_id",
how="left_anti")
display(customers_no_orders)
```

customer_id	first_name	last_name	email	phone_number	address
51	41	2024-03-05	Delivered	39.98	r
52	42	2024-03-06	Shipped	29.99	r
53	43	2024-03-07	Pending	25.0	r
54	44	2024-03-08	Delivered	89.97	r
55	45	2024-03-09	Shipped	49.98	r

4. City-wise total revenue

```
In [ ]: from pyspark.sql.functions import sum

city_total_revenue = customers.join(orders, on="customer_id",
how="inner") \
    .groupBy("city") \
    .agg(sum("total_amount").alias("total_revenue"))
display(city_total_revenue)
```

city	total_revenue
Springfield	69.97
Greenville	169.94
Oakland	69.97
Jacksonville	39.98
San Jose	39.98
Memphis	40.0
Nashville	29.99
San Diego	105.47

5. Average order amount per customer

```
In [ ]: from pyspark.sql.functions import avg

avg_order_amount =
orders.groupBy("customer_id").agg(avg("total_amount").alias("avg_order_a
mount"))
display(avg_order_amount)
```

customer_id	avg_order_amount
14	89.97
16	119.96
35	49.98
3	84.97
8	84.97
12	29.99
20	110.95
27	15.5

6. Customers with more than one order

```
In [ ]: from pyspark.sql.functions import count

customers_multiple_orders =
orders.groupBy("customer_id").agg(count("order_id").alias("order_count"))
).filter("order_count > 1")
display(customers_multiple_orders)
```

customer_id	order_count
3	2
8	2
6	2
9	2
10	2
2	2
4	2
1	2

7. Sort customers by total spend descending

```
In [ ]: from pyspark.sql.functions import sum

customers_total_spend =
orders.groupBy("customer_id").agg(sum("total_amount").alias("total_spend
")).orderBy("total_spend", ascending=False)
display(customers_total_spend)
```

customer_id	total_spend
3	169.94
8	169.94
10	150.95
5	150.95
16	119.96
46	119.96
36	119.96
26	119.96

Exported with [runcell](#) — convert notebooks to HTML or PDF anytime at [runcell.dev](#).